

4

Control Statements: Part 1

OBJECTIVES

- To develop algorithms through the process of top-down, stepwise refinement.
- To use the `if` and `if...else` selection statements.
- To use the `while` repetition statement.
- Counter-controlled repetition and sentinel-controlled repetition.
- To use the increment, decrement and assignment operators.

4.2 Algorithms

- **Algorithms**
 - The actions to execute
 - The order in which these actions execute
- **Program control**
 - Specifies the order in which actions execute in a program.
 - Performs in C++ with control statements.

4.2 Algorithms

- **Example: Rise-and-shine algorithm**
 - (1) Get out of bed**
 - (2) Take off pajamas**
 - (3) Take a shower**
 - (4) Get dressed**
 - (5) Eat breakfast**
 - (6) Carpool to work**

4.3 Pseudocode

- **Pseudocode**

- Artificial, informal language used to develop algorithms
 - Used to think out program before coding
 - Easy to convert into C++ program
- Similar to everyday English
 - Only executable statements
 - No need to declare variables
- Not executed on computers

4.3 Pseudocode

- **Example**

```
1  Prompt the user to enter the first integer
2  Input the first integer
3
4  Prompt the user to enter the second integer
5  Input the second integer
6
7  Add first integer and second integer, store result
8  Display result
```

4.4 Control Structures

- **Sequential execution**
 - Statements execute in sequential order.
- **Transfer of control**
 - Next statement executed is *not* the next one in sequence.

4.4 Control Structures (Cont.)

- **Only three control structures needed**
 - **Three control structures**
 - **Sequence structure**
 - **Programs executed sequentially by default**
 - **Selection structures**
 - **if, if...else, switch**
 - **Repetition structures**
 - **while, do...while, for**

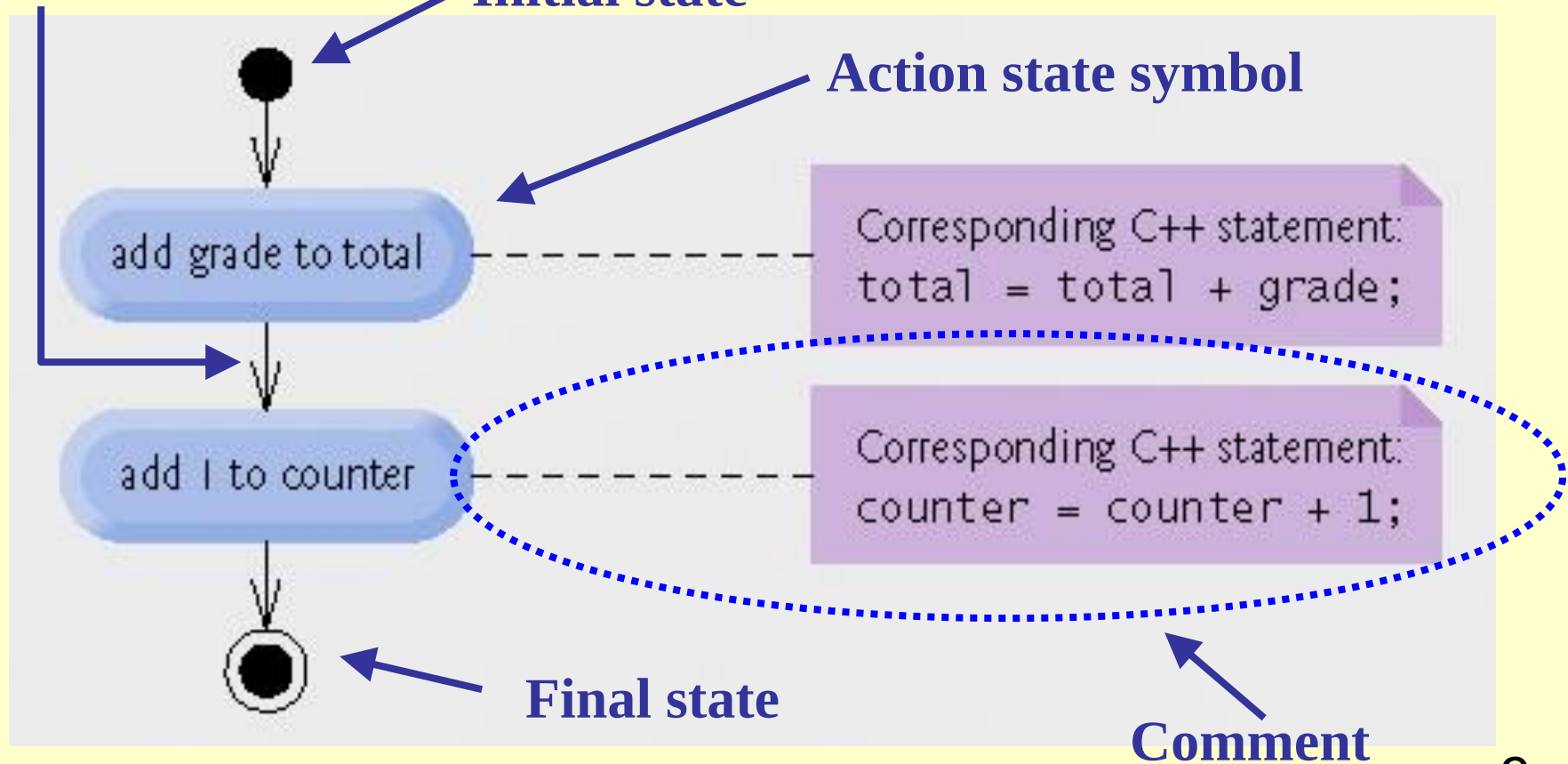
4.4 Control Structures (Cont.)

- UML activity diagram

Flow of activity

Initial state

Action state symbol



4.4 Control Structures (Cont.)

- **Single-entry/single-exit control statements**
 - **Three types of control statements**
 - **Sequence statement**
 - **Selection statements**
 - **Repetition statements**

4.5 if Selection Statement

- **Selection statements**

- Choose among alternative courses of action.
- Pseudocode example

If student's grade is greater than or equal to 60

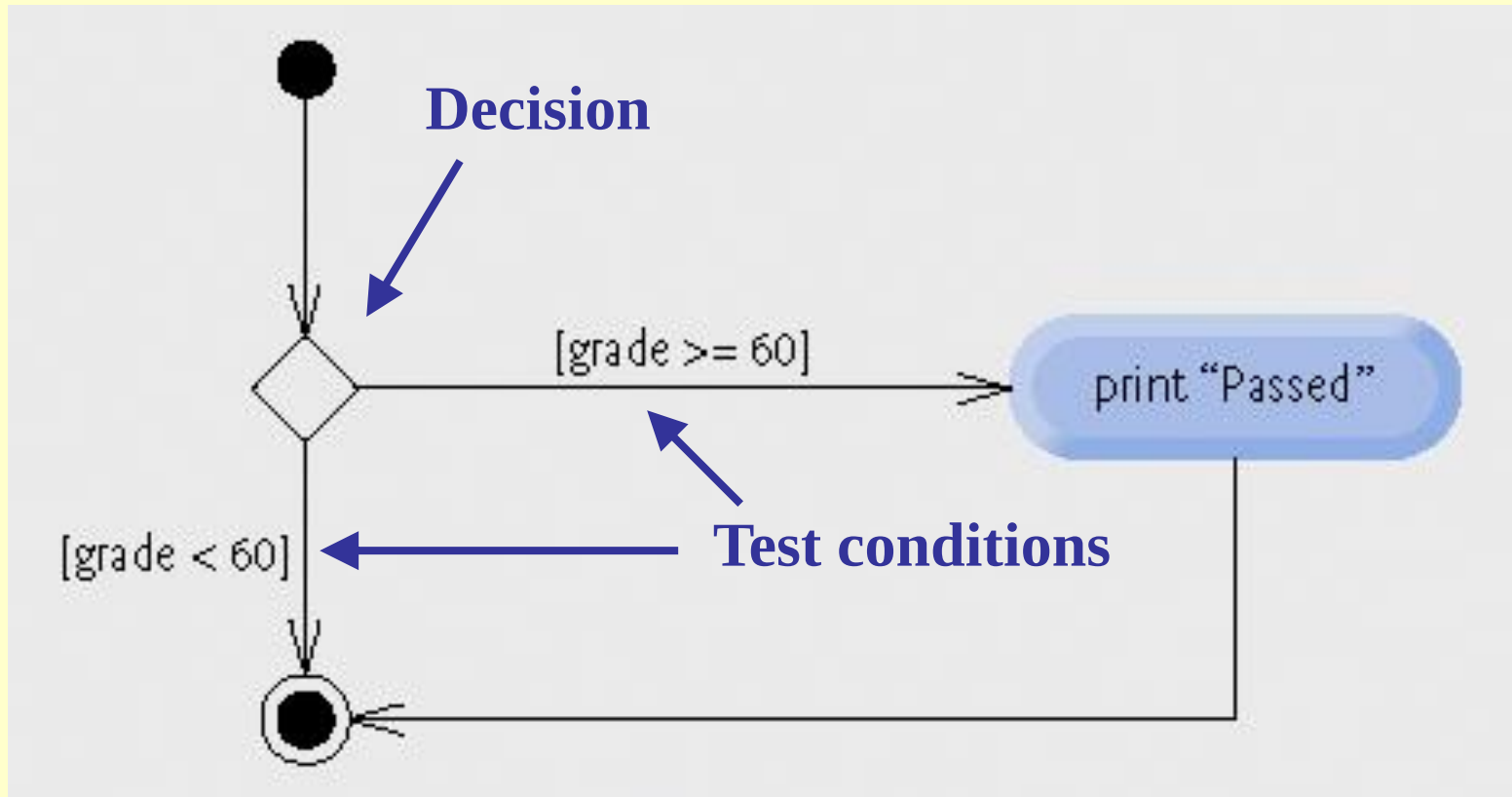
Print "Passed"

- Translation into C++

```
if ( grade >= 60 )  
    cout << "Passed";
```

4.5 if Selection Statement (Cont.)

- **Diamond symbol in UML modeling**



4.6 `if...else` Double-Selection Statement

- **`if`**
 - Performs action if the condition is true.
- **`if...else`**
 - Performs one action if the condition is true, a different action if it is false.

4.6 if...else Double-Selection Statement

- Pseudocode

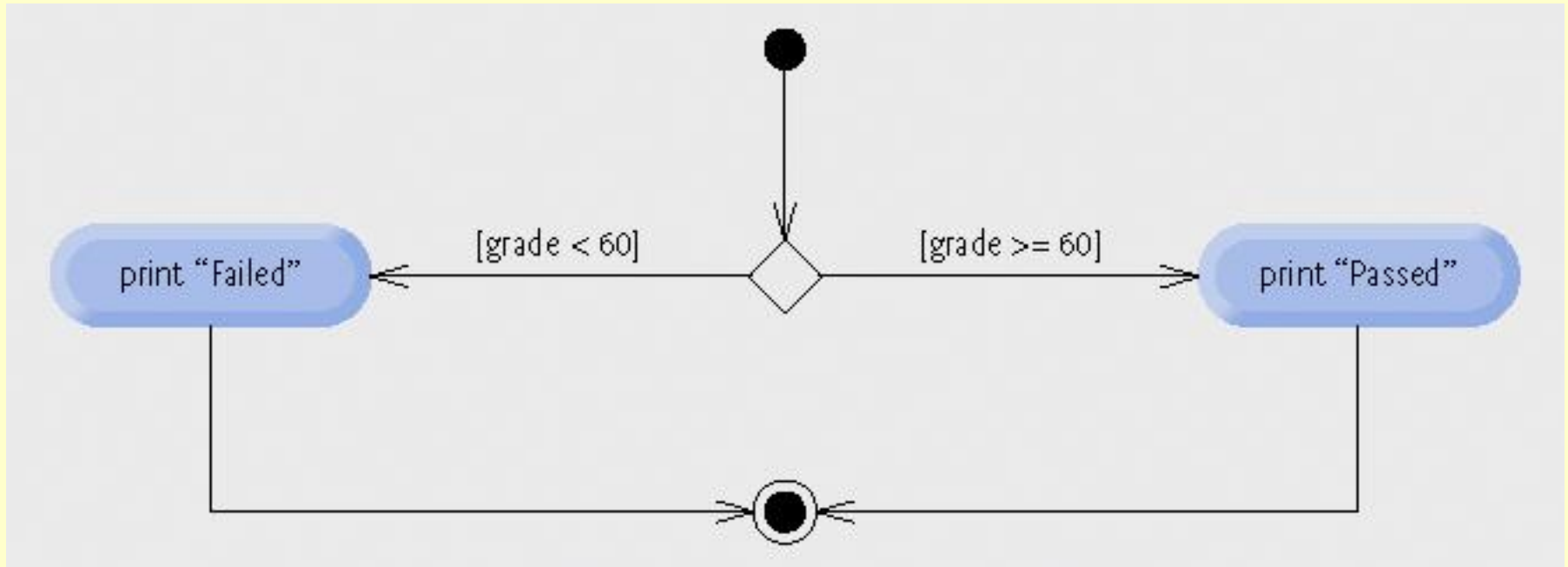
*If student's grade is greater than or equal to 60
 print "Passed"
else
 print "Failed"*

- C++ code

```
if ( grade >= 60 )  
    cout << "Passed";  
else  
    cout << "Failed";
```

4.6 if...else Double-Selection Statement

- **Diamond symbol in UML modeling**



4.6 if...else Double-Selection Statement (Cont.)

- **Ternary conditional operator (?:)**
 - Three arguments (condition, value if true, value if false)
- **Code could be written:**
 - `cout << ( grade >= 60 ? "Passed" : "Failed" );`


Condition


Value if true


Value if false

4.6 if...else Double-Selection Statement (Cont.)

- **Nested if...else statements**
 - One inside another, test for multiple cases
 - Once a condition met, other statements are skipped.
 - Example

If student's grade is greater than or equal to 90

Print "A"

else

If student's grade is greater than or equal to 80

Print "B"

else

If student's grade is greater than or equal to 70

Print "C"

else

If student's grade is greater than or equal to 60

Print "D"

else

Print "F"

4.6 if...else Double-Selection Statement (Cont.)

- **Nested if...else statements (Cont.)**

- Written In C++

```
if ( studentGrade >= 90 )  
    cout << "A";  
else  
    if (studentGrade >= 80 )  
        cout << "B";  
    else  
        if (studentGrade >= 70 )  
            cout << "C";  
        else  
            if ( studentGrade >= 60 )  
                cout << "D";  
            else  
                cout << "F";
```

4.6 if...else Double-Selection Statement (Cont.)

- **Nested if...else statements (Cont.)**

- **Written In C++ (indented differently)**

```
if ( studentGrade >= 90 )  
    cout << "A";  
else if (studentGrade >= 80 )  
    cout << "B";  
else if (studentGrade >= 70 )  
    cout << "C";  
else if ( studentGrade >= 60 )  
    cout << "D";  
else  
    cout << "F";
```

4.6 if...else Double-Selection Statement (Cont.)

- **Dangling-else problem**

- Compiler associates **else** with the immediately preceding **if**.

- Example

```
if ( x > 5 )  
    if ( y > 5 )  
        cout << "x and y are > 5";  
else  
    cout << "x is <= 5";
```

- Compiler interprets as

```
if ( x > 5 )  
    if ( y > 5 )  
        cout << "x and y are > 5";  
else  
    cout << "x is <= 5";
```

4.6 `if...else` Double-Selection Statement (Cont.)

- **Dangling-`else` problem (Cont.)**

- Rewrite with braces (`{}`)

```
if ( x > 5 )
{
    if ( y > 5 )
        cout << "x and y are > 5";
}
else
    cout << "x is <= 5";
```

- Braces indicate that the second `if` statement is in the body of the first and the `else` is associated with the first `if` statement.

4.6 `if...else` Double-Selection Statement (Cont.)

- **Compound statement (Also called a block)**
 - Set of statements within a pair of braces
 - Used to include multiple statements in an `if` body
 - Example

```
if ( studentGrade >= 60 )  
    cout << "Passed.\n";  
else  
{  
    cout << "Failed.\n";  
    cout << "You must take this course again.\n";  
}
```

- Without braces, the following statement always executes:
 cout << "You must take this course again.\n";

Class Work

- **Write the pseudo code and the UML activity diagram to solve the following problem:**

Get an integer from the user. If the integer is a positive integer, the program displays a message “It is greater than 0” to the screen. If the integer is a negative integer, the message should be “It is less than 0”. If the integer is zero, the message should be “It is 0”.

Class Work (Your work)

- **Pseudocode**

Class Work (Your work)

- **UML
activity
diagram**

Class Work (Answer)

- **Pseudocode**

Class Work (Answer)

- **UML
activity
diagram**

4.7 while Repetition Statement

- Repetition statement

- Action repeated while some conditions remain true
- Pseudocode

While there are more items on my shopping list

Purchase next item and cross it off my list

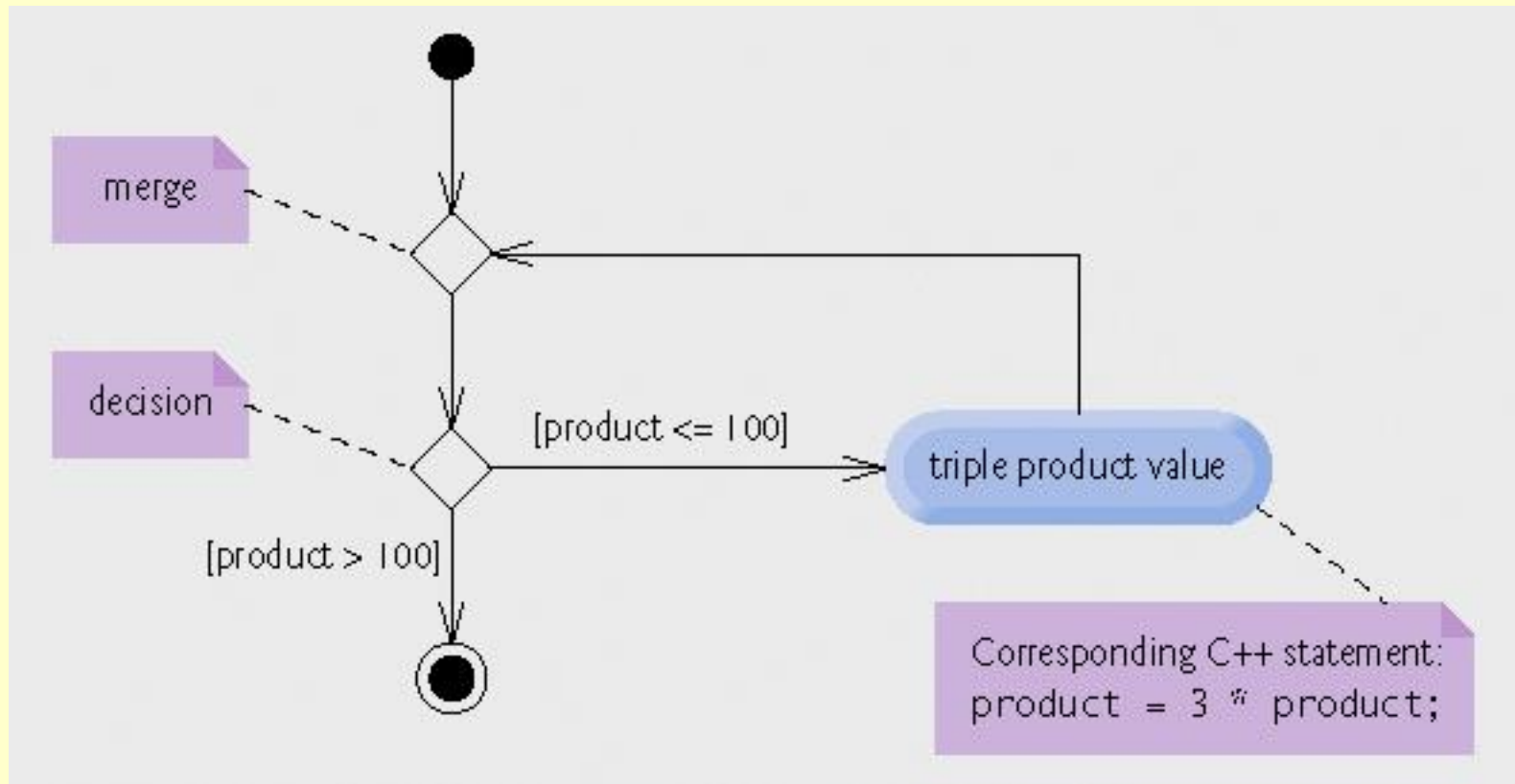
- while loop repeats until condition becomes false.
- Example

```
int product = 3;
```

```
while ( product <= 100 )  
    product = 3 * product;
```

4.7 while Repetition Statement (Cont.)

- **UML merge symbol**



4.8 Formulating Algorithms: Counter-Controlled Repetition

- **Problem statement**

A class of ten students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Calculate and display the total of all student grades and the class average on the quiz.

- **Counter-controlled repetition**

- Loop repeated until counter reaches certain value.
- Also known as definite repetition
 - Number of repetitions known beforehand

4.8 Formulating Algorithms: Counter-Controlled Repetition (Cont.)

- **Counter-controlled repetition (Cont.)**
 - **Counter variable**
 - Use to count.
 - **Example:** a counter variable indicates which of the 10 grades is being entered.
 - **Total variable**
 - Use to accumulate the sum of several values.
 - **Normally initialize to zero beforehand; otherwise, it would include the previous value stored in that memory location.**

4.8 Formulating Algorithms: Counter-Controlled Repetition (Cont.)

- **Pseudocode**

```
1  Set total to zero
2  Set grade counter to one
3
4  While grade counter is less than or equal to ten
5      Prompt the user to enter the next grade
6      Input the next grade
7      Add the grade into the total
8      Add one to the grade counter
9
10 Set the class average to the total divided by ten
11 Print the total of the grades for all students in the class
12 Print the class average
```

```
1 // Fig. 4.8: GradeBook.h
2 // Definition of class GradeBook that determines a class average.
3 // Member functions are defined in GradeBook.cpp
4 #include <string> // program uses C++ standard string class
5 using std::string;
6
7 // GradeBook class definition
8 class GradeBook
9 {
10 public:
11     GradeBook( string ); // constructor initializes course name
12     void setCourseName( string ); // function to set the course name
13     string getCourseName(); // function to retrieve the course name
14     void displayMessage(); // display a welcome message
15     void determineClassAverage(); // averages grades entered by the user
16 private:
17     string courseName; // course name for this GradeBook
18 }; // end class GradeBook
```

Outline

GradeBook.h

(1 of 1)

4.8 Formulating Algorithms: Counter-Controlled Repetition (Cont.)

Line 15: `void determineClassAverage();`

- **Function `determineClassAverage` implements the class average algorithm described by the pseudocode.**

```
1 // Fig. 4.9: GradeBook.cpp
2 // Member-function definitions for class GradeBook that solves the
3 // class average program with counter-controlled repetition.
4 #include <iostream>
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include "GradeBook.h" // include definition of class GradeBook
10
11 // constructor initializes courseName with string supplied as argument
12 GradeBook::GradeBook( string name )
13 {
14     setCourseName( name ); // validate and store courseName
15 } // end GradeBook constructor
16
17 // function to set the course name;
18 // ensures that the course name has at most 25 characters
19 void GradeBook::setCourseName( string name )
20 {
21     if ( name.length() <= 25 ) // if name has 25 or fewer characters
22         courseName = name; // store the course name in the object
23     else // if name is longer than 25 characters
24     { // set courseName to first 25 characters of parameter name
25         courseName = name.substr( 0, 25 ); // select first 25 characters
26         cout << "Name \"" << name << "\" exceeds maximum length (25).\n"
27             << "Limiting courseName to first 25 characters.\n" << endl;
28     } // end if...else
29 } // end function setCourseName
30
```

```

31 // function to retrieve the course name
32 string GradeBook::getCourseName()
33 {
34     return courseName;
35 } // end function getCourseName
36
37 // display a welcome message to the GradeBook user
38 void GradeBook::displayMessage()
39 {
40     cout << "welcome to the grade book for\n" << getCourseName() << "!\n"
41         << endl;
42 } // end function displayMessage
43
44 // determine class average based on 10 grades entered by user
45 void GradeBook::determineClassAverage()
46 {
47     int total; // sum of grades entered by user
48     int gradeCounter; // number of the grade to be entered next
49     int grade; // grade value entered by user
50     int average; // average of grades
51

```

Outline

GradeBook.cpp

(2 of 3)

```

52 // initialization phase
53 total = 0; // initialize total
54 gradeCounter = 1; // initialize loop counter
55
56 // processing phase
57 while ( gradeCounter <= 10 ) // loop 10 times
58 {
59     cout << "Enter grade: "; // prompt for input
60     cin >> grade; // input next grade
61     total = total + grade; // add grade to total
62     gradeCounter = gradeCounter + 1; // increment counter by 1
63 } // end while
64
65 // termination phase
66 average = total / 10; // integer division yields integer result
67
68 // display total and average of grades
69 cout << "\nTotal of all 10 grades is " << total << endl;
70 cout << "Class average is " << average << endl;
71 } // end function determineClassAverage

```

Outline

GradeBook.cpp

(3 of 3)

Outline

fig04_10.cpp

(1 of 1)

```
1 // Fig. 4.10: fig04_10.cpp
2 // Create GradeBook object and invoke its determineClassAverage function.
3 #include "GradeBook.h" // include definition of class GradeBook
4
5 int main()
6 {
7     // create GradeBook object myGradeBook and
8     // pass course name to constructor
9     GradeBook myGradeBook( "CS101 C++ Programming" );
10
11     myGradeBook.displayMessage(); // display welcome message
12     myGradeBook.determineClassAverage(); // find average of 10 grades
13     return 0; // indicate successful termination
14 } // end main
```

Welcome to the grade book for
CS101 C++ Programming

Enter grade: 67
Enter grade: 78
Enter grade: 89
Enter grade: 67
Enter grade: 87
Enter grade: 98
Enter grade: 93
Enter grade: 85
Enter grade: 82
Enter grade: 100

Total of all 10 grades is 846
Class average is 84

4.8 Formulating Algorithms: Counter-Controlled Repetition (Cont.)

Line 66: `average = total / 10;`

- **When dividing two integers, truncation is performed.**
 - Fractional part of the resulting quotient is lost.

4.9 Formulating Algorithms: Sentinel-Controlled Repetition

- **Problem statement**

Develop a class average program that processes grades for an arbitrary number of students each time it is run.

- **Sentinel-controlled repetition**

- Also known as indefinite repetition
- Use a sentinel value
 - Indicates “end of data entry”.
 - A sentinel value cannot also be a valid input value.
 - Also known as a signal, dummy or flag value.

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

- **Top-down, stepwise refinement**
 - Development technique for well-structured programs
 - Top step
 - Single statement conveying overall function of the program
 - Example
 - Determine the class average for the quiz*
 - First refinement
 - Multiple statements using only the sequence structure
 - Example
 - Initialize variables*
 - Input, sum and count the quiz grades*
 - Calculate and print the total of all student grades and the class average*

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

- **Top-down, stepwise refinement (Cont.)**
 - **Second refinement**
 - **Commit to specific variables.**
 - **Use specific control structures.**

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

- Pseudocode

```
1  Initialize total to zero
2  Initialize counter to zero
3
4  Prompt the user to enter the first grade
5  Input the first grade (possibly the sentinel)
6
7  While the user has not yet entered the sentinel
8      Add this grade into the running total
9      Add one to the grade counter
10     Prompt the user to enter the next grade
11     Input the next grade (possibly the sentinel)
12
13 If the counter is not equal to zero
14     Set the average to the total divided by the counter
15     Print the total of the grades for all students in the class
16     Print the class average
17 else
18     Print "No grades were entered"
```

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

- **Floating-point numbers**
 - A real number with a decimal point.
 - C++ provides data types `float` and `double`.
 - `double` numbers can have larger magnitude and finer detail.
 - Called precision
 - Floating-point constant values are treated as `double` values by default.

```
1 // Fig. 4.12: GradeBook.h
2 // Definition of class GradeBook that determines a class average.
3 // Member functions are defined in GradeBook.cpp
4 #include <string> // program uses C++ standard string class
5 using std::string;
6
7 // GradeBook class definition
8 class GradeBook
9 {
10 public:
11     GradeBook( string ); // constructor initializes course name
12     void setCourseName( string ); // function to set the course name
13     string getCourseName(); // function to retrieve the course name
14     void displayMessage(); // display a welcome message
15     void determineClassAverage(); // averages grades entered by the user
16 private:
17     string courseName; // course name for this GradeBook
18 }; // end class GradeBook
```

Outline

GradeBook.h

(1 of 1)

```

1 // Fig. 4.13: GradeBook.cpp
2 // Member-function definitions for class GradeBook that solves the
3 // class average program with sentinel-controlled repetition.
4 #include <iostream>
5 using std::cout;
6 using std::cin;
7 using std::endl;
8 using std::fixed; // ensures that decimal point is displayed
9
10 #include <iomanip> // parameterized stream manipulators
11 using std::setprecision; // sets numeric output precision
12
13 // include definition of class GradeBook from GradeBook.h
14 #include "GradeBook.h"
15
16 // constructor initializes courseName with string supplied as argument
17 GradeBook::GradeBook( string name )
18 {
19     setCourseName( name ); // validate and store courseName
20 } // end GradeBook constructor
21

```

Outline

GradeBook.cpp

(1 of 4)

```

22 // function to set the course name;
23 // ensures that the course name has at most 25 characters
24 void GradeBook::setCourseName( string name )
25 {
26     if ( name.length() <= 25 ) // if name has 25 or fewer characters
27         courseName = name; // store the course name in the object
28     else // if name is longer than 25 characters
29     { // set courseName to first 25 characters of parameter name
30         courseName = name.substr( 0, 25 ); // select first 25 characters
31         cout << "Name \"" << name << "\" exceeds maximum length (25).\n"
32             << "Limiting courseName to first 25 characters.\n" << endl;
33     } // end if...else
34 } // end function setCourseName
35
36 // function to retrieve the course name
37 string GradeBook::getCourseName()
38 {
39     return courseName;
40 } // end function getCourseName
41
42 // display a welcome message to the GradeBook user
43 void GradeBook::displayMessage()
44 {
45     cout << "Welcome to the grade book for\n" << getCourseName() << "!\n"
46         << endl;
47 } // end function displayMessage
48

```

Outline

GradeBook.cpp

(2 of 4)

```

49 // determine class average based on 10 grades entered by user
50 void GradeBook::determineClassAverage()
51 {
52     int total; // sum of grades entered by user
53     int gradeCounter; // number of grades entered
54     int grade; // grade value
55     double average; // number with decimal point for average
56
57     // initialization phase
58     total = 0; // initialize total
59     gradeCounter = 0; // initialize loop counter
60
61     // processing phase
62     // prompt for input and read grade from user
63     cout << "Enter grade or -1 to quit: ";
64     cin >> grade; // input grade or sentinel value
65
66     // loop until sentinel value read from user
67     while ( grade != -1 ) // while grade is not -1
68     {
69         total = total + grade; // add grade to total
70         gradeCounter = gradeCounter + 1; // increment counter
71
72         // prompt for input and read next grade from user
73         cout << "Enter grade or -1 to quit: ";
74         cin >> grade; // input grade or sentinel value
75     } // end while
76

```

Outline

GradeBook.cpp

(3 of 4)

```

77 // termination phase
78 if ( gradeCounter != 0 ) // if user entered at least one grade...
79 {
80     // calculate average of all grades entered
81     average = static_cast< double >( total ) / gradeCounter;
82
83     // display total and average (with two digits of precision)
84     cout << "\nTotal of all " << gradeCounter << " grades entered is "
85         << total << endl;
86     cout << "Class average is " << setprecision( 2 ) << fixed << average
87         << endl;
88 } // end if
89 else // no grades were entered, so output appropriate message
90     cout << "No grades were entered" << endl;
91 } // end function determineClassAverage

```

Outline

GradeBook.cpp

(4 of 4)

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

Line 8: `using std::fixed;`

- **fixed** forces output to print in fixed point format (not scientific notation) and forces trailing zeros and decimal point to print.
 - As opposed to scientific notation (3.1×10^3)

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

```
Line 10: #include <iomanip>  
Line 11: using std::setprecision;
```

- **setprecision** stream manipulator (in header **<iomanip>**) sets numeric output precision.
 - Specifies number of digits of precision to display to the right of the decimal point.
 - Default precision is six digits.

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

```
Line 81: average = static_cast< double >( total ) / gradeCounter;
```

- Calculate average grade using **static_cast< double >** to perform explicit conversion.
- It creates temporary floating-point copy of **total**.
 - Explicit conversion

4.9 Formulating Algorithms: Sentinel-Controlled Repetition (Cont.)

- **Promotion**

- Converting a value (e.g., `int`) to another data type (e.g., `double`) to perform a calculation.
- Implicit conversion
- For example,

```
average = (double) total / gradeCounter;
```

```

1 // Fig. 4.14: fig04_14.cpp
2 // Create GradeBook object and invoke its determineClassAverage function.
3
4 // include definition of class GradeBook from GradeBook.h
5 #include "GradeBook.h"
6
7 int main()
8 {
9     // create GradeBook object myGradeBook and
10    // pass course name to constructor
11    GradeBook myGradeBook( "CS101 C++ Programming" );
12
13    myGradeBook.displayMessage(); // display welcome message
14    myGradeBook.determineClassAverage(); // find average of 10 grades
15    return 0; // indicate successful termination
16 } // end main

```

```

Welcome to the grade book for
CS101 C++ Programming

Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1

Total of all 3 grades entered is 257
Class average is 85.67

```

Outline

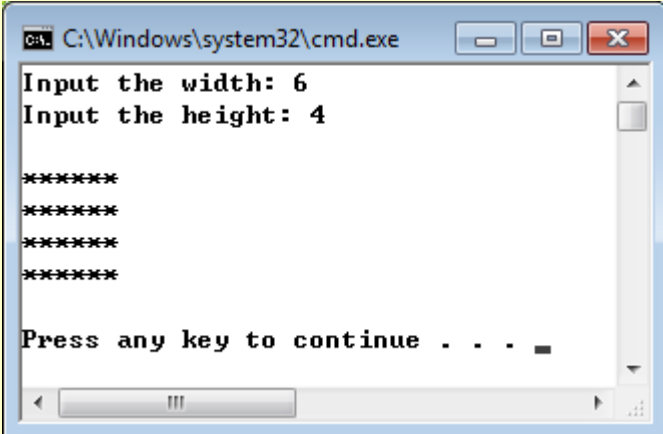
fig04_14.cpp

(1 of 1)

Class Work

- By using the “while” statement, write the program codes to solve the following problem:

The program is to draw a rectangle by using “”. At the beginning, it gets two positive integers (i.e., greater than zero) from the user. One is the width of the rectangle and another one is the height. Then it uses the character “*” to draw the rectangle. A sample output is shown below:*



```
Ca: C:\Windows\system32\cmd.exe
Input the width: 6
Input the height: 4

*****
*****
*****
*****

Press any key to continue . . .
```

Class Work (Your work)

Class Work (Your work)

Class Work (Answer)

4.10 Formulating Algorithms: Nested Control Statement

- **Problem statement**

A college offers a course that prepares students for the state licensing exam for real estate brokers. Last year, ten of the students who completed this course took the exam. The college wants to know how well its students did on the exam.

4.10 Formulating Algorithms: Nested Control Statement

You have been asked to write a program to summarize the results.

You have been given a list of these 10 students.

Next to each name is written a 1 if the student passed the exam or a 2 if the student failed.

4.10 Formulating Algorithms: Nested Control Statement

Your program should analyze the results of the exam as follows:

- 1. Input each test result (i.e., a 1 or a 2). Display the prompting message “Enter result” each time the program requests another test result.*
- 2. Count the number of test results of each type.*
- 3. Display a summary of the test results indicating the number of students who passed and the number who failed.*
- 4. If more than eight students passed the exam, print the message “Raise tuition.”*

4.10 Formulating Algorithms: Nested Control Statement (Cont.)

- **Notice that**
 - **Program processes 10 results.**
 - **Fixed number, use counter-controlled loop**
 - **Each test result is 1 or 2.**
 - **If not 1, assume 2**
 - **Two counters can be used.**
 - **One counts number that passed**
 - **Another counts number that failed**
 - **Must decide whether more than eight students passed.**

4.10 Formulating Algorithms: Nested Control Statement (Cont.)

- **Top level outline**
 - *Analyze exam results and decide whether tuition should be raised*
- **First refinement**
 - *Initialize variables*
Input the ten exam results and count passes and failures
Print a summary of the exam results and decide whether tuition should be raised
- **Second refinement**
 - *Initialize variables*
to
Initialize passes to zero
Initialize failures to zero
Initialize student counter to one

4.10 Formulating Algorithms: Nested Control Statement (Cont.)

- **Second refinement (Cont.)**

- *Input the ten exam results and count passes and failures*
to

While student counter is less than or equal to ten

Prompt the user to enter the next exam result

If the student passed

Add one to passes

Else

Add one to failures

Add one to student counter

4.10 Formulating Algorithms: Nested Control Statement (Cont.)

- **Second refinement (Cont.)**

- *Print a summary of the exam results and decide whether tuition should be raised*

to

Print the number of passes

Print the number of failures

If more than eight students passed

Print “Raise tuition”

4.10 Formulating Algorithms: Nested Control Statement (Cont.)

- Pseudocode

```
1  Initialize passes to zero
2  Initialize failures to zero
3  Initialize student counter to one
4
5  While student counter is less than or equal to 10
6    Prompt the user to enter the next exam result
7    Input the next exam result
8
9    If the student passed
10     Add one to passes
11  Else
12     Add one to failures
13
14  Add one to student counter
15
16 Print the number of passes
17 Print the number of failures
18
19 If more than eight students passed
20  Print "Raise tuition"
```

```
1 // Fig. 4.16: Analysis.h
2 // Definition of class Analysis that analyzes examination results.
3 // Member function is defined in Analysis.cpp
4
5 // Analysis class definition
6 class Analysis
7 {
8 public:
9     void processExamResults(); // process 10 students' examination results
10 }; // end class Analysis
```

Outline

fig04_16.cpp

(1 of 1)

```

1 // Fig. 4.17: Analysis.cpp
2 // Member-function definitions for class Analysis that
3 // analyzes examination results.
4 #include <iostream>
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 // include definition of class Analysis from Analysis.h
10 #include "Analysis.h"
11
12 // process the examination results of 10 students
13 void Analysis::processExamResults()
14 {
15     // initializing variables in declarations
16     int passes = 0; // number of passes
17     int failures = 0; // number of failures
18     int studentCounter = 1; // student counter
19     int result; // one exam result (1 = pass, 2 = fail)
20

```

Outline

fig04_17.cpp

(1 of 2)

```

21 // process 10 students using counter-controlled loop
22 while ( studentCounter <= 10 )
23 {
24     // prompt user for input and obtain value from user
25     cout << "Enter result (1 = pass, 2 = fail): ";
26     cin >> result; // input result
27
28     // if...else nested in while
29     if ( result == 1 ) // if result is 1,
30         passes = passes + 1; // increment passes;
31     else // else result is not 1, so
32         failures = failures + 1; // increment failures
33
34     // increment studentCounter so loop eventually terminates
35     studentCounter = studentCounter + 1;
36 } // end while
37
38 // termination phase; display number of passes and failures
39 cout << "Passed " << passes << "\nFailed " << failures << endl;
40
41 // determine whether more than eight students passed
42 if ( passes > 8 )
43     cout << "Raise tuition " << endl;
44 } // end function processExamResults

```

Outline

fig04_17.cpp

(2 of 2)

```
1 // Fig. 4.18: fig04_18.cpp
2 // Test program for class Analysis.
3 #include "Analysis.h" // include definition of class Analysis
4
5 int main()
6 {
7     Analysis application; // create Analysis object
8     application.processExamResults(); // call function to process results
9     return 0; // indicate successful termination
10 } // end main
```

Outline

fig04_18.cpp

(1 of 2)

Outline

fig04_18.cpp

(2 of 2)

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed 9
Failed 1
Raise tuition
```

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Passed 6
Failed 4
```

4.11 Assignment Operators

- **Assignment expression abbreviations**
 - **Addition assignment operator**
 - **Example**
 - `c = c + 3;` abbreviates to `c += 3;`
- **Statements of the form**
variable = variable operator expression;
can be rewritten as
variable operator = expression;

4.11 Assignment Operators

- **Other assignment operators**

- **d -= 4** (**d = d - 4**)

- **e *= 5** (**e = e * 5**)

- **f /= 3** (**f = f / 3**)

- **g %= 9** (**g = g % 9**)

4.12 Increment and Decrement Operators

- **Increment operator ++**
 - Increments variable by one
 - Example: C++
- **Decrement operator --**
 - Decrement variable by one
 - Example: C--

4.12 Increment and Decrement Operators (Cont.)

- **Pre-increment**

- When the operator is used before the variable (**`++C`** or **`--C`**), the variable is changed and then the expression it is in is evaluated using the new value.

- **Post-increment**

- When the operator is used after the variable (**`C++`** or **`C--`**), the expression the variable is in executes using the old value, then the variable is changed

Outline

fig04_21.cpp

(1 of 1)

```
1 // Fig. 4.21: fig04_21.cpp
2 // Preincrementing and postincrementing.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     int c;
10
11     // demonstrate postincrement
12     c = 5; // assign 5 to c
13     cout << c << endl; // print 5
14     cout << c++ << endl; // print 5 then postincrement
15     cout << c << endl; // print 6
16
17     cout << endl; // skip a line
18
19     // demonstrate preincrement
20     c = 5; // assign 5 to c
21     cout << c << endl; // print 5
22     cout << ++c << endl; // preincrement then print 6
23     cout << c << endl; // print 6
24     return 0; // indicate successful termination
25 } // end main
```

```
5
5
6

5
6
6
```

4.12 Increment and Decrement Operators (Cont.)

- **If `c = 5`, then**
 - **`cout << ++c;`**
 - **`c` is changed to 6**
 - **Then prints out 6**
 - **`cout << c++;`**
 - **Prints out 5 (`cout` is executed before the increment)**
 - **`c` then becomes 6**

4.12 Increment and Decrement Operators (Cont.)

- **When variable is not in an expression**
 - Pre-incrementing and post-incrementing have same effect
 - **Example**
 - `++C;`
`cout << C;`
and
`C++;`
`cout << C;`
are the same

Until Next Time

- **Review Chapter 4**
- **Preview Chapter 5**